concurrently without any blocking until the commit call where the serializability checks are done.
1. Log application acquires an exclusive lock on the repository, which blocks snapshotting.

```
mermaid
sequenceDiagram
    autonumber
    gRPC Server->>+Transaction Middleware: Request
    Transaction Middleware->>+Transaction Manager: Begin
    Transaction Manager->>+Transaction: Open Transaction
    participant Repository
    critical Shared Lock on Repository
        Transaction->>+Snapshot: Create Snapshot
    end
    Transaction->>Transaction Manager: Transaction Opened
    Transaction Manager->>Transaction Middleware: Begun
    Transaction Middleware->>+RPC Handler: Rewritten Request
    RPC Handler->>+git update-ref: Update References
    git update-ref->>Snapshot: Prepare
    Snapshot->>git update-ref: Prepared
    git update-ref->>Snapshot: Commit
    Snapshot->>git update-ref: Committed
    git update-ref->>+Reference Transaction Hook: Invoke
    Reference Transaction Hook->>Transaction: Capture Updates
    Transaction->>Reference Transaction Hook: OK
    Reference Transaction Hook->>-git update-ref: OK
    git update-ref->>-RPC Handler: References Updated
    RPC Handler->>-Transaction Middleware: Success
    Transaction Middleware->>Transaction: Commit
    Transaction->>Transaction Manager: Commit
    critical Serializability Check
        Transaction Manager->>Transaction Manager: Verify Transaction
    end
    Transaction Manager->>Repository: Log Transaction
    Repository->>Transaction Manager: Transaction Logged
    Transaction Manager->>Transaction: Committed
    Transaction->>Snapshot: Remove Snapshot
    deactivate Snapshot
    Transaction->>-Transaction Middleware: Committed
    Transaction Middleware->>-gRPC Server: Success
    critical Exclusive Lock on Repository
        Transaction Manager->>-Repository: Apply Transaction
    end
```

Future opportunities

Expose transactions to clients

Once Gitaly internally has transactions, the next natural step is to expose them to the

clients. For example, Rails could run multiple operations in a single transaction. This would extend the ACID guarantees to the clients, which would solve a number of issues:

- The clients would have ability to commit transactions atomically. Either all changes they make are performed or none are.
- The operations would automatically be guarded against races through the serializability guarantees.

For Gitaly maintainers, extending the transactions to clients enables reducing our API surface. Gitaly has multiple RPCs that perform the same operations. For example, references are updated in multiple RPCs. This increases complexity. If the clients can begin, stage changes, and commit a transaction, we can have fewer, more fine grained RPCs. For example, UserCommitFiles could be modeled with more fine grained commands as:

- Begin
- WriteBlob
- WriteTree
- WriteCommit
- UpdateReference
- Commit

This makes the API composable because the clients can use the single-purpose RPCs to compose more complex operations. This might lead to a concern that each operation requires multiple RPC calls, increasing the latency due to roundtrips. This can be mitigated by providing an API that allows for batching commands.

Other databases provide these features through explicit transactions and a query language.

Continuous backups with WAL archiving

Incremental backups are currently prohibitively slow because they must always compute the changes between the previous backup and the current state of the repository. Because all writes to a partition go through the write-ahead log, it's possible to stream the write-ahead log entries to incrementally back up the repository. For more information, see Repository Backups.

Raft replication

The transactions provide serializability on a single partition. The partition's write-ahead log can be replicated using a consensus algorithm such as Raft. Because Raft guarantees linearizability for log entry commits, and the transaction manager ensures serializability of transactions prior to logging them, all operations across the replicas get serializability guarantees. For more information, see epic 8903.

Alternative solutions

No alternatives have been proposed to the transaction management. The current state of squashing concurrency- and write interruption-related bugs one by one is not scalable.

Snapshot isolation with reftables

Our preliminary designs for snapshot isolation relied on reftables, a new reference backend in Git. Reftables have been a work in progress for years and there doesn't seem to be a clear timeline for when they'll actually land in Git. They have a number of shortcomings compared to the proposed solution here:

- Reftables only cover references in a snapshot. The snapshot design here covers the complete repository, most importantly object database content.
- Reftables would require heavy integration as each Git invocation would have to be wired to read the correct version of a reftable. The file system -based snapshot design here requires no changes to the existing Git invocations.
- The design here gives a complete snapshot of a repository, which enables running multiple RPCs on the same transaction because the transaction's state is stored on the disk during the transaction. Each RPC is able to read the transaction's earlier writes but remain isolated from other transactions. It's unclear how this would be implemented with reftables, especially when it comes to object isolation. This is needed if we want to extend the transaction interface to the clients.
- The snapshots are independent from each other. This reduces synchronization because each transaction can proceed with staging their changes without being blocked by any other transactions. This enables optimistic locking for better performance.

Reftables are still useful as a more efficient reference backend but they are not needed for snapshot isolation.

SECTION: engineering/architecture/design-documents/gitlab_agent_deployments/_index.md

{{< engineering/design-document-header >}}

Summary

As part of the GitLab Kubernetes Dashboard epic,
users want to view and manage their resources deployed by GitLab agent For Kubernetes.
Users should be able to interact with the resources through GitLab UI, such as Environment Index/Details page.

This blueprint describes how the association is established and how these domain models interact with each other.

Motivation

Goals

- The proposed architecture can be used in GitLab Kubernetes Dashboard.
- The proposed architecture can be used in Organization-level Environment dashboard.
- The cluster resources and events can be visualized per GitLab Environment. An environment-specific view scoped to the resources managed either directly or indirectly by a deployment commit.
- Support both GitOps mode and CI Access mode.

Non-Goals

- The design details of GitLab Kubernetes Dashboard and Organization-level Environment dashboard.
- Support Environment/Deployment features that rely on GitLab CI/CD pipelines, such as Protected Environments, Deployment Approvals, Deployment safety, and Environment rollback. These features are already available in CI Access mode, however, it's not available in GitOps mode.

Proposal

Overview

- GitLab Environment and GitLab agent For Kubernetes have 1-to-1 relationship.
- GitLab Environment tracks all resources produced by the connected agent. This includes not only resources written in manifest files but also subsequently generated resources (for example, Pods created by Deployment manifest file).
- GitLab Environment renders dependency graph, such as Deployment => ReplicaSet => Pod. This is for providing ArgoCD-style resource view.
- GitLab Environment has the Resource Health status that represents a summary of resource statuses, such as Healthy, Progressing or Degraded.

```
mermaid
flowchart LR
  subgraph Kubernetes["Kubernetes"]
    subgraph ResourceGroupProduction["Production"]
      direction LR
      ResourceGroupProductionService(["Service"])
      ResourceGroupProductionDeployment(["Deployment"])
      ResourceGroupProductionPod1(["Pod1"])
      ResourceGroupProductionPod2(["Pod2"])
    end
    subgraph ResourceGroupStaging["Staging"]
      direction LR
      ResourceGroupStagingService(["Service"])
      ResourceGroupStagingDeployment(["Deployment"])
      ResourceGroupStagingPod1(["Pod1"])
      ResourceGroupStagingPod2(["Pod2"])
    end
  end

  subgraph GitLab
    subgraph Organization
      subgraph Project
        environment1["production environment"]
        environment2["staging environment"]
      end
    end
  end
```

```
  environment1 --- ResourceGroupProduction
  environment2 --- ResourceGroupStaging
  ResourceGroupProductionService -.- ResourceGroupProductionDeployment
  ResourceGroupProductionDeployment -.- ResourceGroupProductionPod1
  ResourceGroupProductionDeployment -.- ResourceGroupProductionPod2
  ResourceGroupStagingService -.- ResourceGroupStagingDeployment
  ResourceGroupStagingDeployment -.- ResourceGroupStagingPod1
  ResourceGroupStagingDeployment -.- ResourceGroupStagingPod2
```

Existing components and relationships

- GitLab Project and GitLab Environment have 1-to-many relationship.
- GitLab Project and Agent have 1-to-many direct relationship. Only one project can own a specific agent.
- GitOps mode
  - GitLab Project and Agent do NOT have many-to-many indirect relationship yet. This will be supported in Manifest projects outside of the Agent configuration project.
- CI Access mode
  - GitLab Project and Agent have many-to-many indirect relationship. The project owning the agent can share the access with the other proejcts. (NOTE: Technically, only running jobs inside the project are allowed to access the cluster due to job-token authentication.)

Issues

- GitLab Environment should have ID of GitLab agent For Kubernetes as the foreign key.
- GitLab Environment should have parameters how to group resources in the associated cluster, for example, namespace, lable and inventory-id (GitOps mode only) can passed as parameters.
- GitLab Environment should be able to fetch all relevant resources, including both default resource kinds and other Custom Resources.
- GitLab Environment should be aware of dependency graph.
- GitLab Environment should be able to compute Resource Health status from the associated resources.

Example

This is an example of how the architecture works in push-based deployment.
The feature is documented here as CI access mode.

```
mermaid
flowchart LR
  subgraph ProductionKubernetes["Production Kubernetes"]
    subgraph ResourceGroupProductionFrontend["Production"]
      direction LR
      ResourceGroupProductionFrontendService(["Service"])
      ResourceGroupProductionFrontendDeployment(["Deployment"])
      ResourceGroupProductionFrontendPod1(["Pod1"])
      ResourceGroupProductionFrontendPod2(["Pod2"])
    end
    subgraph ResourceGroupProductionBackend["Staging"]
```

```
    direction LR
    ResourceGroupProductionBackendService(["Service"])
    ResourceGroupProductionBackendDeployment(["Deployment"])
    ResourceGroupProductionBackendPod1(["Pod1"])
    ResourceGroupProductionBackendPod2(["Pod2"])
  end
  subgraph ResourceGroupProductionPrometheus["Monitoring"]
    direction LR
    ResourceGroupProductionPrometheusService(["Service"])
    ResourceGroupProductionPrometheusDeployment(["Deployment"])
    ResourceGroupProductionPrometheusPod1(["Pod1"])
    ResourceGroupProductionPrometheusPod2(["Pod2"])
  end
end

subgraph GitLab
  subgraph Organization
    subgraph OperationGroup
      subgraph AgentManagementProject
        AgentManagementAgentProduction["Production agent"]
        AgentManagementManifestFiles["Kubernetes Manifest Files"]
        AgentManagementEnvironmentProductionPrometheus["production prometheus environment"]
        AgentManagementPipelines["CI/CD pipelines"]
      end
    end
    subgraph DevelopmentGroup
      subgraph FrontendAppProject
        FrontendAppCode["VueJS"]
        FrontendDockerfile["Dockerfile"]
      end
      subgraph BackendAppProject
        BackendAppCode["Golang"]
        BackendDockerfile["Dockerfile"]
      end
      subgraph DeploymentProject
        DeploymentManifestFiles["Kubernetes Manifest Files"]
        DeploymentPipelines["CI/CD pipelines"]
        DeploymentEnvironmentProductionFrontend["production frontend environment"]
        DeploymentEnvironmentProductionBackend["production backend environment"]
      end
    end
  end
end

DeploymentEnvironmentProductionFrontend --- ResourceGroupProductionFrontend
DeploymentEnvironmentProductionBackend --- ResourceGroupProductionBackend
AgentManagementEnvironmentProductionPrometheus --- ResourceGroupProductionPrometheus
ResourceGroupProductionFrontendService -.- ResourceGroupProductionFrontendDeployment
ResourceGroupProductionFrontendDeployment -.- ResourceGroupProductionFrontendPod1
ResourceGroupProductionFrontendDeployment -.- ResourceGroupProductionFrontendPod2
```

```
  ResourceGroupProductionBackendService -.- ResourceGroupProductionBackendDeployment
  ResourceGroupProductionBackendDeployment -.- ResourceGroupProductionBackendPod1
  ResourceGroupProductionBackendDeployment -.- ResourceGroupProductionBackendPod2
  ResourceGroupProductionPrometheusService -.- ResourceGroupProductionPrometheusDeployment
  ResourceGroupProductionPrometheusDeployment -.- ResourceGroupProductionPrometheusPod1
  ResourceGroupProductionPrometheusDeployment -.- ResourceGroupProductionPrometheusPod2
  AgentManagementAgentProduction -- Shared with --- DeploymentProject
  DeploymentPipelines -- "Deploy" --> ResourceGroupProductionFrontend
  DeploymentPipelines -- "Deploy" --> ResourceGroupProductionBackend
  AgentManagementPipelines -- "Deploy" --> ResourceGroupProductionPrometheus
```

Further details

Multi-Project Deployment Pipelines

The microservice project setup can be improved by Multi-Project Deployment Pipelines:

- Deployment Project can behave as the shared deployment engine for any upstream application projects and environments.
- Environments can be created within the application projects. It gives more visibility of environments for developers.
- Deployment Project can be managed under Operator group. More segregation of duties.
- Users don't need to set up RBAC to restrict CI/CD jobs.
- This is especially helpful for dynamic environments like review apps.

mermaid

```
flowchart LR
  subgraph ProductionKubernetes["Production Kubernetes"]
    subgraph ResourceGroupProductionFrontend["Frontend"]
      direction LR
      ResourceGroupProductionFrontendService(["Service"])
      ResourceGroupProductionFrontendDeployment(["Deployment"])
      ResourceGroupProductionFrontendPod1(["Pod1"])
      ResourceGroupProductionFrontendPod2(["Pod2"])
    end
    subgraph ResourceGroupProductionBackend["Backend"]
      direction LR
      ResourceGroupProductionBackendService(["Service"])
      ResourceGroupProductionBackendDeployment(["Deployment"])
      ResourceGroupProductionBackendPod1(["Pod1"])
      ResourceGroupProductionBackendPod2(["Pod2"])
    end
    subgraph ResourceGroupProductionPrometheus["Monitoring"]
      direction LR
      ResourceGroupProductionPrometheusService(["Service"])
      ResourceGroupProductionPrometheusDeployment(["Deployment"])
      ResourceGroupProductionPrometheusPod1(["Pod1"])
      ResourceGroupProductionPrometheusPod2(["Pod2"])
    end
```

```
 end

 subgraph GitLab
  subgraph Organization
    subgraph OperationGroup
      subgraph DeploymentProject
        DeploymentAgentProduction["Production agent"]
        DeploymentManifestFiles["Kubernetes Manifest Files"]
        DeploymentEnvironmentProductionPrometheus["production prometheus environment"]
        DeploymentPipelines["CI/CD pipelines"]
      end
    end
    subgraph DevelopmentGroup
      subgraph FrontendAppProject
        FrontendDeploymentPipelines["CI/CD pipelines"]
        FrontendEnvironmentProduction["production environment"]
      end
      subgraph BackendAppProject
        BackendDeploymentPipelines["CI/CD pipelines"]
        BackendEnvironmentProduction["production environment"]
      end
    end
  end
end

 FrontendEnvironmentProduction --- ResourceGroupProductionFrontend
 BackendEnvironmentProduction --- ResourceGroupProductionBackend
 DeploymentEnvironmentProductionPrometheus --- ResourceGroupProductionPrometheus
 ResourceGroupProductionFrontendService -.- ResourceGroupProductionFrontendDeployment
 ResourceGroupProductionFrontendDeployment -.- ResourceGroupProductionFrontendPod1
 ResourceGroupProductionFrontendDeployment -.- ResourceGroupProductionFrontendPod2
 ResourceGroupProductionBackendService -.- ResourceGroupProductionBackendDeployment
 ResourceGroupProductionBackendDeployment -.- ResourceGroupProductionBackendPod1
 ResourceGroupProductionBackendDeployment -.- ResourceGroupProductionBackendPod2
 ResourceGroupProductionPrometheusService -.- ResourceGroupProductionPrometheusDeployment
 ResourceGroupProductionPrometheusDeployment -.- ResourceGroupProductionPrometheusPod1
 ResourceGroupProductionPrometheusDeployment -.- ResourceGroupProductionPrometheusPod2
 FrontendDeploymentPipelines -- "Trigger downstream pipeline" --> DeploymentProject
 BackendDeploymentPipelines -- "Trigger downstream pipeline" --> DeploymentProject
 DeploymentPipelines -- "Deploy" --> ResourceGroupProductionFrontend
 DeploymentPipelines -- "Deploy" --> ResourceGroupProductionBackend
```

Design and implementation details

Associate Environment with Agent

Users can explicitly set a GitLab agent For Kubernetes to a GitLab Environment in setting UI. Frontend will use this associated agent for authenticating/authorizing the user access, which is described in a latter section.

We need to adjust the readclusteragent permission in DeclarivePolicy for supporting agents shared by an external project (also known as the Agent management project).

Fetch resources through useraccess

When user visits an environment page, GitLab frontend fetches an environment via GraphQL. Frontend additionally fetches the associated agent-ID and namespace.

Here is an example of GraphQL query:

```
graphql
{
  project(fullPath: "group/project") {
    id
    environment(name: "<environment-name>") {
      slug
      kubernetesNamespace
      clusterAgent {
        id
        name
        project {
          name
        }
      }
    }
  }
}
```

GitLab frontend authenticate/authorize the user access with browser cookie. If the access is forbidden, frontend shows an error message that You don't have access to an agent that deployed to this environment. Please contact agent administrator if you are allowed in "useraccess" in agent config file. See <troubleshooting-doc-link>.

After the user gained access to the agent, GitLab frontend fetches specific Resource kinds (for example, Deployment, Pod) in the Kubernetes with the following parameters:

- namespace ... {environment.kubernetesNamespace}

If no resources are found, this is likely that the users have not embedded these lables into their resources. In this case, frontend shows an warning message There are no resources found for the environment. Do resources have GitLab preserved labels? See <troubleshooting-doc-link>.

Dependency graph

- GitLab frontend uses Owner References to idenfity the dependencies between resources. These are embedded in resources as metadata.ownerReferences field.
- For the resoruces that don't have owner references, we can use Well-Known Labels, Annotations and Taints as complement. for example, EndpointSlice doesn't have metadata.ownerReferences, but has kubernetes.io/service-name as a reference to the parent Service resource.

Health status of resources

- GitLab frontend computes the status summary from the fetched resources. Something similar to ArgoCD's Resource Health for example, Healthy, Progressing, Degraded and Suspended. The formula is TBD.

SECTION: engineering/architecture/design-documents/gitlab_ci_events/_index.md

{{< engineering/design-document-header >}}

Summary

In order to unlock innovation and build more value, GitLab is expected to be the center of automation related to DevSecOps processes. We want to transform GitLab into a programming environment, that will make it possible for engineers to model various workflows on top of CI/CD pipelines. Today, users must create custom automation around webhooks or scheduled pipelines to build required workflows.

In order to make this automation easier for our users, we want to build a powerful CI/CD eventing system, that will make it possible to run pipelines whenever something happens inside or outside of GitLab.

A typical use-case is to run a CI/CD job whenever someone creates an issue, posts a comment, changes a merge request status from "draft" to "ready for review" or adds a new member to a group.

To build that new technology, we should:

1. Emit many hierarchical events from within GitLab in a more advanced way than we do it today.
1. Make it affordable to run this automation, that will react to GitLab events, at scale.
1. Provide a set of conventions and libraries to make writing the automation easier.

Goals

While "GitLab Events Platform" aims to build new abstractions around emitting events in GitLab, "GitLab CI Events" blueprint is about making it possible to:

1. Define a way in which users will configure when an event emitted will result in a CI pipeline being run.
1. Describe technology required to match subscriptions with events at GitLab.com scale and beyond.
1. Describe technology we could use to reduce the cost of running automation jobs significantly.

Proposal

Decisions

- 001: Use hierarchical events

Requirements

Any accepted proposal should take in consideration the following requirements and characteristics:

1. Defining events should be done in separate files.
   - If we define all events in a single file, then the single file gets too complicated and hard to
     maintain for users. Then, users need to separate their configs with the include keyword again and we end up
     with the same solution.
   - The structure of the pipelines, the personas and the jobs will be different depending on the events being
     subscribed to and the goals of the subscription.
1. A single subscription configuration file should define a single pipeline that is created when an event is triggered.
   - The pipeline config can include other files with the include keyword.
   - The pipeline can have many jobs and trigger child pipelines or multi-project pipelines.
1. The events and handling syntax should use the existing CI config syntax where it is pragmatic to do so.
   - It'll be easier for users to adapt. It'll require less work to implement.
1. The event subscription and emiting events should be performant, scalable, and non blocking.
   - Reading from the database is usually faster than reading from files.
   - A CI event can potentially have many subscriptions.
     This also includes evaluating the right YAML files to create pipelines.
   - The main business logic (e.g. creating an issue) should not be affected
     by any subscriptions to the given CI event (e.g. issue created).
1. The CI events design should be implemented in a maintainable and extensible way.
   - If there is a issues/create event, then any new event (mergerequest/created) can be added without
     much effort.
   - We expect that many events will be added. It should be trivial for developers to
     register domain events (e.g. 'issue closed') as GitLab-defined CI events.
   - Also, we should consider the opportunity of supporting user-defined CI events long term (e.g. 'order shipped').

Options

For now, we have technical 5 proposals;

1. Proposal 1: Using the .gitlab-ci.yml file
   Based on;
   - GitLab CI Workflows PoC
   - PoC NPM CI events
1. Proposal 2: Using the rules keyword
   Highly inefficient way.

1. Proposal 3: Using the .gitlab/ci/events folder
   Involves file reading for every event.
1. Proposal 4: Creating events via a CI config file
   Separate configuration files for defininig events.
1. Proposal 5: Combined proposal
   Combination of all of the proposals listed above.

SECTION: engineering/architecture/design-documents/gitlab_ci_events/proposal-1-using-the-gitlab-ci-file.md

Currently, we have two proof-of-concept (POC) implementations:

- GitLab CI Workflows PoC
- PoC NPM CI events

They both have similar ideas;

1. Find a new CI Config syntax to define pipeline events.

   Example 1:

   yaml
   workflow:
     events:
       - events/package/published

    or

   workflow:
     on:
       - events/package/published

   Example 2:

   yaml
   spec:
     on:
       - events/package/published
       - events/package/removed
      on:
        package: [published, removed]
   ---
   dosomething:
     script: echo "Hello World"

1. Upsert a workflow definition to the database when new configuration gets

pushed.
1. Match subscriptions and publishers whenever something happens at GitLab.

Discussion

1. How to efficiently detect changes to the subscriptions?
1. How do we handle differences between workflows / events / subscriptions on different branches?
1. Do we need to upsert subscriptions on every push?

SECTION: engineering/architecture/design-documents/gitlab_ci_events/proposal-2-using-the-rules-keyword.md

Can we do it with our current rules system?

```
yaml
workflow:
  rules:
    - events: ["package/"]

testpackagepublished:
  script: echo testing published package
  rules:
    - events: ["package/published"]

testpackageremoved:
  script: echo testing removed package
  rules:
    - events: ["package/removed"]
```

1. We don't upsert subscriptions to the database.
1. We'll have a single worker which runs when something happens in GitLab.
1. The worker just tries to create a pipeline with the correct parameters.
1. Pipeline runs when rules subsystem finds a job to run.

Challenges

1. For every defined event run, we need to enqueue a new pipeline creation worker.
1. Creating pipelines and selecting builds to run is a relatively expensive operation
1. This will not work on GitLab.com scale.

SECTION: engineering/architecture/design-documents/gitlab_ci_events/proposal-3-using-the-gitlab-ci-events-folder.md

In this proposal we want to create separate files for each group of events. We

can define events in the following format:

yaml
 .gitlab/ci/events/package-published.yml

```
spec:
  events:
    - name: package/published

job1:
  script: echo "Hello World"

job2:
  script: echo "Hello World"

job-for-package-published:
  script: echo "Hello World"
  rules:
    - if: $[[ inputs.event ]] == "package/published"
```

When an event happens;

1. We'll enqueue a new job for the event.
1. The job will search for the event file in the .gitlab/ci/events folder.
1. The job will run Ci::CreatePipelineService for the event file.

 Problems & Questions

1. For every defined event run, we need to enqueue a new job.
1. Every event-job will need to search for files.
1. This would be only for the project-scope events.
1. This will not work for GitLab.com scale.

SECTION: engineering/architecture/design-documents/gitlab_ci_events/proposal-4-creating-events-via-ci-files.md

Each project can have its own configuration file for defining subscriptions to
events. For example, .gitlab-ci-event.yml. In this file, we can define events
in the following format:

yaml
```
events:
  - package/published
  - issue/created
```

When this file is changed in the project repository, it is parsed and the

events are created, updated, or deleted. This is highly similar to
Proposal 1 except that we don't need
to track pipeline creations every time.

1. Upsert events to the database when .gitlab-ci-event.yml gets updated.
1. Create inline reactions to events in code to trigger pipelines.

Filtering jobs

We can filter jobs by using the rules keyword. For example:

yaml
testpackagepublished:
  script: echo testing published package
  rules:
    - events: ["package/published"]

testpackageremoved:
  script: echo testing removed package
  rules:
    - events: ["package/removed"]

Otherwise, we can make it work either a CI variable;

yaml
testpackagepublished:
  script: echo testing published package
  rules:
    - if: $CIEVENT == "package/published"

testpackageremoved:
  script: echo testing removed package
  rules:
    - if: $CIEVENT == "package/removed"

or an input like in the Proposal 3:

yaml
spec:
  inputs:
    event:
      default: push

---

testpackagepublished:
  script: echo testing published package
  rules:

```
    - if: $[[ inputs.event ]] == "package/published"

testpackageremoved:
  script: echo testing removed package
  rules:
    - if: $[[ inputs.event ]] == "package/removed"
```

Challenges

1. This will not work on GitLab.com scale.

SECTION: engineering/architecture/design-documents/gitlab_ci_events/proposal-5-combined-proposal.md

In this proposal we have separate files for cohesive groups of events. The
files are being included into the main .gitlab-ci.yml configuration file.

yaml
my/events/packages.yaml

```
spec:
  events:
    - events/package/published
    - events/audit/package/
  inputs:
    env:

rspec-on-push:
  script: bundle exec rspec
```

yaml
.gitlab/workflows/mergerequests.yml

```
spec:
  events:
    - events/mergerequest/push

smoke-test:
  script: bundle exec rspec --smoke
```

SECTION: engineering/architecture/design-documents/gitlab_ci_events/decisions/001_hierarchical_events.md

Context

We did some brainstorming in an issue
with multiple use-cases for running CI pipelines based on subscriptions to CI events. The pattern of using hierarchical events emerged, it is clear that events may be grouped together by type or by origin.

For example:

```
yaml
annotate:
  on: issue/created
  script: ./annotate $[[ event.issue.id ]]

summarize:
  on: issue/closed
  script: ./summarize $[[ event.issue.id ]]
```

When making this decision we didn't focus on the syntax yet, but the grouping of events seems to be useful in majority of use-cases.

We considered making it possible for users to subscribe to multiple events in a group at once:

```
yaml
audit:
  on: events/gitlab/gitlab-org/audit/
  script: ./audit $[[ event.operation.name ]]
```

The implication of this is that events within the same groups should share same fields / schema definition.

Decision

Use hierarchical events: events that can be grouped together and that will share the same fields following a stable contract. For example: all issue events will contain issue.iid field.

How we group events has not been decided yet, we can either do that by labeling or grouping using path-like syntax.

Consequences

The implication is that we will need to build a system with stable interface describing events' payload and / or schema.

Alternatives

An alternative is not to use hierarchical events, and making it necessary to subscribe to every event separately, without giving users any guarantess around

common schema for different events. This would be especially problematic for events that naturally belong to some group and users expect a common schema for, like audit events.

SECTION: engineering/architecture/design-documents/gitlab_data_exploration/_index.md

Summary

This design document addresses the challenge of data exploration and querying across GitLab's diverse data landscape. Users currently face significant obstacles when trying to access and derive insights from GitLab data, as information is spread across multiple sources including PostgreSQL, ClickHouse, GraphQL, and REST APIs.

Each of these data sources requires different query strategies and presents varying schemas, relationships, and performance characteristics. This fragmentation creates substantial friction in the data exploration process, requiring users to understand multiple systems and query languages to access the information they need.

GitLab currently lacks a unified querying interface and visualization tool that can work consistently across these different data sources. This absence forces users to switch between different interfaces and tools, making it difficult to create comprehensive dashboards that combine data from multiple sources or to explore relationships between different types of data.

The goal is to simplify how users interact with GitLab data, enabling them to discover meaningful insights about their GitLab usage and business performance without needing to understand the underlying differences between data sources. By addressing this challenge, we aim to unlock valuable data that is currently difficult or impossible for most users to access due to technical complexity.

Motivation

The ability for users to explore their GitLab data and derive meaningful business insights is increasingly important as organizations rely on data-driven decision making. However, several challenges currently prevent users from effectively exploring and understanding their GitLab data.

This initiative serves both our external customers and internal GitLab team members. External users need data insights to optimize their DevSecOps practices and demonstrate the value of GitLab within their organizations. Simultaneously, GitLab's own product, engineering, and customer success teams rely on these same data exploration capabilities to understand product usage, improve features, and support customers effectively. The unified data exploration architecture described in this document will benefit both audiences, creating a consistent and valuable experience regardless of whether the user is a customer or a GitLab team member.

Challenges

Data Source Fragmentation

GitLab data resides across multiple data sources, each with different access patterns:

- PostgreSQL databases store transactional application data
- Elasticsearch/OpenSearch databases contain denormalized data for efficient searching
- ClickHouse databases contain analytical and time-series data
- GraphQL endpoints provide structured API access to application data
- REST APIs offer additional interfaces to various data sets

Each of these sources has evolved independently, resulting in different query requirements, data models, and performance characteristics. This fragmentation forces users to understand each system separately to explore their data effectively.

Furthermore, even data sources of the same class can exhibit significant variations; for example, different REST API endpoints may vary in their filtering capabilities, query methods, performance impacts, and response formats.

Query Language Inconsistency

Users currently need to employ different query approaches depending on the data source:

- GraphQL has its own query structure
- REST APIs use various parameter-based filtering mechanisms
- PostgreSQL and ClickHouse databases can be queried by the end-user only via the GraphQL and REST APIs
- Filters and operators vary across endpoints even within the same API type

This inconsistency creates a steep learning curve for users who need to access data across multiple sources and requires specialized knowledge of each system's querying capabilities.

Schema and Data Model Disparities

Beyond the query language differences, there are fundamental inconsistencies in how data is structured:

- Field naming conventions vary across systems
- Entity relationships are modeled differently
- Data granularity differs (for instance detailed records vs. aggregated data)

These disparities make it difficult to establish meaningful connections between related data points that exist in different systems; limiting users' ability to gain a complete picture of their information.

User Experience Friction

The current state creates significant friction in the data exploration process:

- Not all data sources have a data exploration UI or a query language.
- Users must often switch between multiple tools or interfaces to access different data sources.
- Non-technical users face significant barriers to exploring data on their own.

- There's often a lack of transparency about how the data was queried and transformed, creating trust issues with the presented information. GitLab team members have to spend significant time explaining to customers how data is retrieved and displayed, trying to build trust in the tool.

This friction discourages data exploration and limits the insights users can derive from their GitLab data.

Performance and Resource Challenges

Different data sources have varying performance characteristics:

- Some queries may be resource-intensive and could impact system performance
- Query optimization strategies differ across data sources
- Performance can vary dramatically for similar queries against different data sources

These challenges make it difficult to provide a consistently responsive exploration experience across all data types.

Opportunities for Unified Data Exploration

Despite these challenges, there are significant opportunities to simplify and enhance how users interact with their GitLab data:

- A unified data exploration interface would simplify access to critical insights, improve feature adoption, and unlock data that is currently inaccessible to most users due to technical complexity
- Standardizing query patterns could unlock new cross-source analytics capabilities
- Enabling export, sharing, and embedding of query results would allow users to incorporate insights across other GitLab pages and workflows
- A standardized input and output would not only benefit users, but also other systems integrating with it or used to generate queries or interpret results, such as an AI assistant.

A well-designed data exploration architecture would not only address the current pain points but also establish a foundation for more advanced analytics capabilities in the future.

Examples of User Needs

The unified data exploration system would enable users to answer questions such as:

For Engineering Teams:

- "As an engineering team, every day during standup, we want to look at one dashboard that tells us how we are doing in relation to our DevSecOps flow. We will need to see:
  - Table with current open MRs
  - Count of critical and high vulnerabilities introduced in the last x days
  - Issues assigned to our current milestone (or label/status, whatever pivot the team would want to look at their current work)
  - Recent pipeline failures or slow jobs
  - DORA metrics like deployment frequency and lead time for changes"

For Product Managers:

- "As an GitLab PM, I want to track internal usage and engagement metrics for my feature"
- "How are users navigating through my feature's workflow?"
- "What is the adoption rate of new capabilities we've released?"
- "How does feature usage correlate with other activities in the product?"

These examples illustrate how users need to combine data from multiple sources (issues, MRs, vulnerabilities, pipelines, analytics events) in a single cohesive view·something that's currently difficult to achieve.

Goals

- Create a unified data exploration experience across GitLab's diverse data sources
- Simplify the process of querying data for both technical and non-technical users
- Standardize the filtering interface to work consistently regardless of underlying data source
- Enable exporting of queries result across GitLab pages
- Standardize query result by leveraging existing dashboard framework components
- Ensure appropriate performance for data exploration queries across different data sources
- Maintain proper security controls and respect user permissions across all data sources
- Facilitate integration with GitLab Duo to enhance data exploration capabilities

Non-Goals

- Creating a dashboard framework or new visualizations components - We will use existing framework visualization components rather than creating new ones
- Fixing inconsistencies between existing API's and datasources - We won't directly resolve inconsistencies in existing APIs and data sources. Rather, we're creating an interface layer that abstracts these differences away from the use

Proposal

This proposal outlines a solution to the data exploration challenges identified earlier. We aim to create a unified, intuitive interface that allows users to query and visualize data from multiple sources within GitLab, regardless of where the data resides or how it's structured.

The solution consists of two main components:

1. A standardized and simplified query system - An extension of GitLab Query Language (GLQL) to work across multiple data sources and with enhanced querying capabilities
2. A unified data exploration UI - A standardized interface for constructing queries, viewing results, and creating visualizations

A Standardized And Simplified Query System

At the core of our solution is a standardized query system that builds upon the existing GitLab Query Language (GLQL).

Why GLQL is the ideal foundation

GLQL provides a robust foundation for our standardized query system:

1. Reduced cognitive load - Users will learn one query system instead of multiple query languages or methods, improving productivity and making it easier to adopt
2. Enable queries from multiple sources - Data from different sources can be easily queried in a consistent and meaningful way
3. Provide consistent results - Uniform filtering and output formats across data sources
4. User-friendly syntax - GLQL was designed with simplicity and readability in mind
5. Extensible by design - The GLQL architecture separates concerns between parsing, execution, transformation, and presentation and was built with extensibility in mind
6. Production-proven - The current implementation is already in use for merge requests and work item queries
7. AI-Powered Exploration: GLQL's structured syntax makes it an ideal foundation for integration with GitLab Duo. This will enable natural language queries where users can ask questions in plain English and have them automatically translated to GLQL, making data exploration accessible to a wider audience.

Extending GLQL to support multiple data sources

The proposed approach includes:

1. Extend GLQL to support new data sources - Extend the type field to include more data sources

```
plaintext
query: project = "team-project" AND type = Vulnerability AND severity in ("critical",
"high") AND created > -7d
fields: status, severity, description

query: project = "team-project" AND type = Pipeline AND (status = failed OR duration > 60)
AND updated > -3d
fields: id, name, status, duration, updatedAt

query: project = "team-project" AND type = AiMetric
fields: codeSuggestionsShownCount, codeSuggestionsAcceptedCount, duoUsedCount
```

2. Improved querying capabilities - Increase the query language power by adding support for:

- OR operators (currently only AND supported)
- Mathematical operators
- Aggregating functions
- Sorting

```
plaintext
query: project = "team-project" AND type = Issue AND updated > -3d
fields: count() AS "Total Issues", sum(weight) AS "Total Weight"
groupby: state
sortby: count()
```

Enabling users to perform more advanced queries is a fundamental requirement for a functional and useful data exploration system and query language. Being able to aggregate or compute data (for example, calculate the MR throughput over time of a project) provides much more value than just pulling down some plain data (for example, a list of filtered MRs).

Adding some of these capabilities to GLQL is already tracked in https://gitlab.com/gitlab-org/gitlab/-/issues/511954.

3. Enhanced display options - Expand the display attribute to support visualization types that might be more appropriate for new datasources, such as charts:

plaintext
Line chart display
display: chart
charttype: line
xaxis: createdat
yaxis: count

Custom visualization component
display: custom
displayid: 'ai-impact-table'

Some early explorations have been done in https://gitlab.com/gitlab-org/gitlab/-/issues/482782.

4. Support large dataset - As the current implementation of GLQL only supports returning a single page of data, we need to expand that to fully support pagination.

5. Support querying data for multiple projects and groups - Currently GLQL is limited to queries scoped to a single project or a single group. Fetching data from multiple projects would require executing separate queries and combine the results, while losing context, pagination and sorting. We need to expand that to support multiple groups and projects, so that users are allowed to access all data across their instance or organisation.

Lastly, adopting GLQL for dashboard data exploration could also enable easy exporting and sharing of dashboards/visualizations across other GitLab pages, further enhancing the platform's data exploration capabilities.

Moving GLQL to the backend

A critical architectural change is moving GLQL execution from the frontend to the backend, creating a new separate API to query any GitLab data with a consistent query and filter language.

Currently the GLQL compiler is only translating GLQL queries into GraphQL, and relies on the consumer to execute the query. This restricts GLQL capabilities as a language to the same limitations imposed by GraphQL, making it hard or not possible to implement features such as OR operators, nested subqueries, aggregation or mathematical functions. This has been discussed in depth in https://gitlab.com/gitlab-org/gitlab/-/issues/546157.

In addition, given the inconsistent GraphQL schema for filters and data types, adding new data sources to GLQL is not straightforward and requires handling ad-hoc cases during the transpilation stage. This potentially makes it harder for teams to onboard their data sources to GLQL, hence limiting its impact and usability.

A possible solution to this is to move the GLQL compiler to the backend, and transpiling queries into an ad-hoc intermediate format (like JSON), following a more consistent schema, that can be used to query data directly through Rails finders, avoiding going down the GraphQL path altogether.

Proof of concepts:

- Multiple outputs support + FFI wrapper
- Move GLQL to backend

Moving GLQL to the backend would provide the following advantages:

- A single entry point for querying GitLab data, with centralized access control and consistent querying interface
- Enables to extend the query language with advanced capabilities (OR operations, aggregation, math functions)
- Opportunity for opening up GLQL to satellite services, such as IDE extensions, third party services and APIs. For example it could enable using GLQL in a CLI tool like glab, or rendering the output of a GLQL query in an email notification.
- A simplified frontend implementation, with the backend as single source of truth
- Queries can be executed closer to the data
- Opportunity for optimisations at the backend level, such as increased concurrency of queries execution
- Promotes GLQL from just a compiler to being a full platform, where you can ask a query and get the appropriate data back. Previously this responsibility lay with the consumer, but now GLQL would own it.

In addition, having the GLQL Rust compiler also allows the same parser to be shared by both frontend and backend contexts:

- Backend: Query parsing and full query execution against data sources
- Frontend: Syntax validation and immediate feedback without query execution

This is also in line with ~devops::plan future plans: https://gitlab.com/groups/gitlab-org/-/epics/15834, thus opening up opportunities for collaboration.

This standardized query system when built with an extended GLQL architecture and shift to the backend, will provide the foundation for a powerful and consistent data exploration experience across all GitLab data sources.

A unified data exploration UI

The unified data exploration UI will provide a cohesive, intuitive interface for users to query, analyze, and visualize data from GitLab's diverse data sources. The UI will serve both technical users who may prefer writing GLQL queries directly and non-technical users who need a

simplified visual interface to build reports and dashboards.

The interface should include the following main building blocks:

1. Query Construction Area

- Toggle between text-based GLQL editor and visual query building
- Syntax highlighting, autocompletion, and error detection for GLQL query editor
- Easily accessible filters
- Visual query builder with intuitive components for non-technical users
- Schema browser showing available fields and data types
- Query templates and saved queries library
- AI-assisted building

2. Results Display

- Tabular view for raw data exploration
- Visualization canvas for charts and graphs
- Preview of query results while building the query
- Pagination controls for large result sets
- AI-assisted results interpretation

3. Visualization Controls

- Visualization type selector (charts, tables, metrics, markdown, custom views)
- Configuration panel for the selected visualization

4. Action Toolbar

- Run query button
- Save query button
- Share/export options
- Queries history
- Dashboard integration options

5. Context Panel

- Metadata about the current query (for instance execution time, rows returned, or some performance metrics)
- Documentation and help resources

This unified interface will integrate seamlessly with the standardized query system, leveraging GLQL's capabilities while presenting them in an accessible way to all users regardless of their technical expertise.

Whilst the above are the main building blocks that we think are necessary to build the data exploration interface, proper UX research will be required to turn this into a proper design.

<!--
Design and implementation details

This section should contain enough information that the specifics of your change are understandable. This may include API specs (though not always required) or even code snippets. If there's any ambiguity about HOW your proposal will be implemented, this is the place to discuss them.

If you are not sure how many implementation details you should include in the document, the rule of thumb here is to provide enough context for people to understand the proposal. As you move forward with the implementation, you may need to add more implementation details to the document, as those may become valuable context for important technical decisions made along the way. A document is also a register of such technical decisions. If a technical decision requires additional context before it can be made, you probably should document this context in a document. If it is a small technical decision that can be made in a merge request by an author and a maintainer, you probably do not need to document it here. The impact a technical decision will have is another helpful information - if a technical decision is very impactful, documenting it, along with associated implementation details, is advisable.

If it's helpful to include workflow diagrams or any other related images. Diagrams authored in GitLab flavored markdown are preferred. In cases where that is not feasible, images should be placed under images/ in the same directory as the index.md for the proposal.
-->

Alternative Solutions

1. Build a GLQL-powered data explorer without any architectural changes to GLQL

We can rely on GLQL as it stands now and add some data sources that would serve as a good first use case.

- Pros: No major architecture changes needed. Would still allow us to get some of the benefits of using GLQL.
- Cons: Binds GLQL to the limitations of GraphQL for more advanced queries (OR, subqueries, aggregations, math expressions). We could also face a roadblock if the current limitations of GLQL prove difficult to work with.

2. Build a data explorer without GLQL, relying on a visual query builder

We can reuse some of the current implementation of data explorer, or even what was built for the observability metrics query builder, and extend it to work with multiple data sources, filters, and queries. It might require ad-hoc frontend datasource adapters to process queries for each data type.

- Pros: No dependency on GLQL, no major architecture changes needed.
- Cons: Building a visual query builder that works for all data sources is not trivial due to schema and filtering differences. Embedding/sharing queries would also be less straightforward than a GLQL-based solution.

3. Limit the scope of the data explorer and provide a set of prebuilt queries/visualizations

for creating custom dashboards

We could reduce the scope of the data explorer and provide an extensible set of predefined queries for each data type that each team would curate.

- Pros: No dependency on GLQL, no major architecture changes needed, no need to build a query builder that fits all cases.
- Cons: Less freedom for users to explore their data. Each feature team will need to decide on a set of predefined queries for users.

SECTION: engineering/architecture/design-documents/gitlab_duo_rag/_index.md

{{< engineering/design-document-header >}}

RAG is an application architecture used to provide knowledge to a large language model that doesn't exist in its training set, so that it can use that knowledge to answer user questions. To learn more about RAG, see RAG for GitLab.

Goals of this blueprint

This blueprint aims to drive a decision for a RAG solution for GitLab Duo on self-managed, specifically for shipping GitLab Duo with access to GitLab documentation. We outline three potential solutions, including PoCs for each to demonstrate feasibility for this use case.

Constraints

- The solution must be viable for self-managed customers to run and maintain
- The solution must be shippable in 1-2 milestones <!-- I don't actually know that this is true, just adding an item for time constraint -->
- The solution should be low-lock-in, since we are still determining our long term technical solution(s) for RAG at GitLab

Proposals for GitLab Duo Chat RAG for GitLab documentation

The following solutions have been proposed and evaluated for the GitLab Duo Chat for GitLab documentation use case:

- Vertex AI Search
- Elasticsearch
- PostgreSQL with PGVector extension

You can read more about how each evaluatoin was conducted in the links above.

Chosen solution

Vertex AI Search is going to be implemented due to the low lock-in and being able to reach customers quickly. It could be moved over to another solution in the future.

SECTION: engineering/architecture/design-documents/gitlab_duo_rag/elasticsearch.md

For more information on Elasticsearch and RAG broadly, see the Elasticsearch article in RAG at GitLab.

Retrieve GitLab Documentation

A proof of concept was done to switch the documentation embeddings from being stored in the embedding database to being stored on Elasticsearch.

Synchronizing embeddings with data source

The same procedure used by PostgreSQL can be followed to keep the embeddings up to date in Elasticsearch.

Retrieval

To get the nearest neighbours, the following query can be executed an index containing the embeddings:

```
ruby
{
  "knn": {
    "field": vectorfieldcontainingembeddings,
    "queryvector": embeddingforquestion,
    "k": limit,
    "numcandidates": numberofcandidatestocompare
  }
}
```

Requirements to get to self-managed

- Productionalize the PoC MR
- Get more self-managed instances to install Elasticsearch by shipping GitLab with Elasticsearch. Elastic gave their approval to ship with the free license. The work required for making it easy for customers to host Elasticsearch is more than 2 milestones.

SECTION: engineering/architecture/design-documents/gitlab_duo_rag/postgresql.md

Retrieve GitLab Documentation

PGVector is currently being used for the retrieval of relevant documentation for GitLab Duo chat's RAG.

A separate embedding database runs alongside geo and main which has the pg-vector extension

installed and contains embeddings for GitLab documentation.

- Statistics (as of January 2024):
  - Data type: Markdown written in natural language (Unstructured)
  - Data access level: Green (No authorization required)
  - Data source: https://gitlab.com/gitlab-org/gitlab/-/blob/master/doc
  - Data size: 147 MB in vertexgitlabdocs. 2194 pages.
  - Service: https://docs.gitlab.com/ (source repo
  - Example of user input: "How do I create an issue?"
  - Example of expected AI-generated response: "To create an issue:\n\nOn the left sidebar, select Search or go to and find your project.\n\nOn the left sidebar, select Plan > Issues, and then, in the upper-right corner, select New issue."

Synchronizing embeddings with data source

Here is the overview of synchronizing process that is currently running in GitLab.com:

1. Load documentation files of the GitLab instance. i.e. doc//.md.
1. Compare the checksum of each file to detect an new, update or deleted documents.
1. If a doc is added or updated:
   1. Split the docs with the following strategy:
      - Text splitter: Split by new lines (\n). Subsequently split by 100~1500 chars.
   1. Bulk-fetch embeddings of the chunks from textembedding-gecko model (768 dimensions).
   1. Bulk-insert the embeddings into the vertexgitlabdocs table.
   1. Cleanup the older embeddings.
1. If a doc is deleted:
   1. Delete embeddings of the page.

As of today, there are 17345 rows (chunks) on vertexgitlabdocs table on GitLab.com.

For Self-managed instances, we serve embeddings from AI Gateway and GCP's Cloud Storage, so the above process can be simpler:

1. Download an embedding package from Cloud Storage through AI Gateway API.
1. Bulk-insert the embeddings into the vertexgitlabdocs table.
1. Delete older embeddings.

We generate this embeddings package before GitLab monthly release.
Sidekiq cron worker automatically renews the embeddings by comparing the embedding version and the GitLab version.
If it's outdated, it will download the new embedding package.

Going further, we can consolidate the business logic between SaaS and Self-managed by generating the package every day (or every grpd deployment).
This is to reduce the point of failure in the business logic and let us easily reproduce an issue that reported by Self-managed users.

Here is the current table schema:

sql

```
CREATE TABLE vertexgitlabdocs (
    id bigint NOT NULL,
    createdat timestamp with time zone NOT NULL,
    updatedat timestamp with time zone NOT NULL,
    version integer DEFAULT 0 NOT NULL,                    -- For replacing the
old embeddings by new embeddings (e.g. when doc is updated)
    embedding vector(768),                                 -- Vector
representation of the chunk
    url text NOT NULL,
    content text NOT NULL,                                 -- Chunked data
    metadata jsonb NOT NULL,                               -- Additional metadata
e.g. page URL, file name
    CONSTRAINT check2e35a254ce CHECK ((charlength(url) <= 2048)),
    CONSTRAINT check93ca52e019 CHECK ((charlength(content) <= 32768))
);

CREATE INDEX indexvertexgitlabdocsonversionandmetadatasourceandid ON vertexgitlabdocs USING
btree (version, ((metadata ->> 'source'::text)), id);
CREATE INDEX indexvertexgitlabdocsonversionwhereembeddingisnull ON vertexgitlabdocs USING btree
(version) WHERE (embedding IS NULL);
```

Retrieval

After the embeddings are ready, GitLab-Rails can retrieve chunks in the following steps:

1. Fetch embedding of the user input from textembedding-gecko model (768 dimensions).
1. Query to vertexgitlabdocs table for finding the nearest neighbors. e.g.:

```
  sql
  SELECT
  FROM vertexgitlabdocs
  ORDER BY vertexgitlabdocs.embedding <=> '[vectors of user input]'              -- nearest
neighbors by cosine distance
  LIMIT 10
```

Requirements to get to self-managed

All instances of GitLab have postgres running but allowing instances to administer a separate database for embeddings or combining the embeddings into the main database would require some effort which spans more than a milestone.

SECTION: engineering/architecture/design-documents/gitlab_duo_rag/vertex_ai_search.md

Retrieve GitLab Documentation

- Statistics (as of January 2024):

- Date type: Markdown (Unstructured) written in natural language
- Date access level: Green (No authorization required)
- Data source: https://gitlab.com/gitlab-org/gitlab/-/blob/master/doc
- Data size: approx. 56,000,000 bytes. 2194 pages.
- Service: https://docs.gitlab.com/ (source repo)
- Example of user input: "How do I create an issue?"
- Example of expected AI-generated response: "To create an issue:\n\nOn the left sidebar, select Search or go to and find your project.\n\nOn the left sidebar, select Plan > Issues, and then, in the upper-right corner, select New issue."

The GitLab documentation is the SSoT service to serve GitLab documentation for SaaS (both GitLab.com and Dedicated) and Self-managed.
When a user accesses to a documentation link in GitLab instance,
they are redirected to the service since 16.0 (except air-gapped solutions).
In addition, the current search backend of docs.gitlab.com needs to transition to Vertex AI Search. See this issue (GitLab member only) for more information.

We introduce a new semantic search API powered by Vertex AI Search for the documentation tool of GitLab Duo Chat.

Setup in Vertex AI Search

We create a search app for each GitLab versions.
These processes will likely be automated in the GitLab Documentation project
by CI/CD pipelines.

1. Create a new Bigquery table e.g. gitlab-docs-latest or gitlab-docs-v16.4
1. Download documents from repositories (e.g. gitlab-org/gitlab/doc, gitlab-org/gitlab-runner/docs, gitlab-org/omnibus-gitlab/doc).
1. Split them by Markdown headers and generate metadata (e.g. URL and title).
1. Insert rows into the Bigquery table.
1. Create a search app

See this notebook for more implementation details.
The data of the latest version will be refreshed by a nightly build with Data Store API.

AI Gateway API

API design is following the existing patterns in AI Gateway.

plaintext
POST /v1/search/docs

json
```
{
  "type": "search",
  "metadata": {
    "source": "GitLab EE",
    "version": "16.3"         // Used for switching search apps for older GitLab instances
```

```
  },
  "payload": {
    "query": "How can I create an issue?",
    "params": {              // Params for Vertex AI Search
      "pagesize": 10,
      "filter": "",
    },
    "provider": "vertex-ai"
  }
}
```

The response will include the search results. For example:

```
json
{
  "response": {
    "results": [
      {
        "id": "d0454e6098773a4a4ebb613946aadd89",
        "content": "\nTo create an issue from a group:  \n1. On the left sidebar, ...",
        "metadata": {
          "Header1": "Create an issue",
          "Header2": "From a group",
          "url": "https://docs.gitlab.com/ee/user/project/issues/createissues.html"
        }
      }
    ]
  },
  "metadata": {
    "provider": "vertex-ai"
  }
}
```

See SearchRequest and SearchResponse for Vertex AI API specs.

Proof of Concept

- GitLab-Rails MR
- AI Gateway MR
- Vertex AI Search service
- Google Colab notebook
- Demo video (Note: In this video, Website URLs are used as data source).

Evaluation score

Here is the evaluation scores generated by Prompt Library.

|Setup|correctness|comprehensiveness|readability|evaluatingmodel|

|---|---|---|---|---|
|New (w/ Vertex AI Search)|3.7209302325581382|3.6976744186046511|3.9069767441860455|claude-2|
|Current (w/ Manual embeddings in GitLab-Rails and PgVector)|3.7441860465116279|3.6976744186046511|3.9767441860465116|claude-2|

<details>

<summary>Dataset</summary>

- Input Bigquery table: dev-ai-research-0e2f8974.duochatexternal.documentationinputv1
- Output Bigquery table:
  - dev-ai-research-0e2f8974.duochatexternalresults.smdoctoolvertexaisearch
  - dev-ai-research-0e2f8974.duochatexternalresults.smdoctoollegacy
- Command: promptlib duo-chat eval --config-file /eval/data/config/duochatevalconfig.json

</details>

Estimated Time of Completion

- Milestone N:
  - Setup in Vertex AI Search with CI/CD automation.
  - Introduce /v1/search/docs endpoint in AI Gateway.
  - Updates the retrieval logic in GitLab-Rails.
  - Feature flag clean up.

Total milestones: 1

SECTION: engineering/architecture/design-documents/gitlab_duo_with_amazon_q/_index.md

{{< engineering/design-document-header >}}

Amazon Q, GitLab Dedicated, and Duo are partnering to execute a superior developer experience with a bundled offering.

This design doc is located at https://gitlab.com/gitlab-org/architecture/gitlab-duo-with-amazon-q/design-doc.

Who

<!-- vale gitlab.Spelling = NO -->

Who	Role
Sam Goldstein	Director of Engineering, Engineering DRI
Grzegorz Bizon	Distinguished Engineer - Technical Lead
Stan Hu	Engineering Fellow
Jessie Young	Principal Engineer
Imre Farkas	Staff Backend Engineer

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-documents/gitlab_events_platform/_index.md

{{< engineering/desig